LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 6: Generate Data Otomatis dan Authentication dengan Laravel



Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S
NIM: 707012530010
Kelas: D4SIKC-49-04
Mata Kuliah: Pengembangan Aplikasi Berbasis Web

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
D4 SISTEM INFORMASI KOTACERDAS

Tujuan

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami cara kerja authentication, session, hashing password, dan middleware pada Laravel.
2. Memasang Laravel Breeze dan memakai fitur register, login, serta logout yang dibuat otomatis.
3. Menerapkan authentication pada project LaporBanjir dari Modul 4 tanpa menghilangkan fitur pelaporan warga.
4. Membuat akun petugas dummy dengan seeder dan Faker berlocale id_ID, lalu memastikan password tersimpan sebagai hash.
5. Melindungi halaman daftar laporan dengan middleware auth dan menampilkan menu navigasi sesuai status login.

Alat dan Bahan

Praktikum ini dikerjakan pada laptop dengan spesifikasi berikut.

Perangkat Keras

- Laptop dengan prosesor 13th Gen Intel Core i7-13620H.
- RAM 16 GB (terbaca 15,7 GB).
- Sistem operasi Windows 11 Home Single Language build 10.0.26200, versi 25H2.

Perangkat Lunak

- XAMPP 8.2.12, di dalamnya terdapat Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1.
- Composer 2.10.3 sebagai dependency manager PHP.
- Laravel Framework 12.70.0.
- Laravel Breeze 2.4.2 sebagai starter kit authentication.
- Node.js 24.11.1 dan NPM 11.6.2 untuk mengompilasi asset Breeze.
- Vite 7.3.7 dan Tailwind CSS 3.4.19 sebagai build tool dan kerangka tampilan.
- Visual Studio Code sebagai teks editor.
- Google Chrome untuk menjalankan aplikasi di browser.

- Git 2.53.0 untuk version control.
- Database MySQL/MariaDB bernama db_laporbanjir.

C. Langkah-Langkah Praktikum

Praktikum ini melanjutkan project LaporBanjir dari Modul 4. Project disimpan di folder praktikumta6 dan memakai database db_laporbanjir yang sama, jadi data laporan lama tetap bisa dipakai. Saya memasang Laravel Breeze untuk menangani authentication, lalu menyesuaikan route, layout, dan view LaporBanjir agar berjalan di atas Breeze. Bagian ini mencatat urutan pekerjaan yang saya lakukan.

C.1 Menyiapkan Database dan Project

Langkah pertama adalah menyalakan Apache dan MySQL pada XAMPP Control Panel. Database db_laporbanjir sudah ada dari Modul 4 dan berisi tabel users, laporan, sessions, cache, jobs, serta migrations. Karena tabelnya sudah dibuat, saya tidak memulai dari database kosong.

Koneksi database diatur pada file .env di folder utama project. Bagian yang saya ubah adalah konfigurasi DB seperti berikut.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_laporbanjir
DB_USERNAME=root
DB_PASSWORD=

```

Nilai DB_CONNECTION diisi mysql, DB_HOST 127.0.0.1, DB_PORT 3306, DB_DATABASE db_laporbanjir, DB_USERNAME root, dan DB_PASSWORD kosong mengikuti pengaturan default XAMPP. Saya juga mengubah APP_NAME menjadi LaporBanjir dan APP_FAKER_LOCALE menjadi id_ID supaya nama dummy mengikuti locale Indonesia.

Setelah konfigurasi itu, saya menjalankan php artisan config:clear dan php artisan migrate. Perintah migrate menampilkan pesan Nothing to migrate karena semua migration dari Modul 4 sudah tercatat di tabel migrations. Struktur tabel users tetap sama, termasuk kolom password bertipe varchar(255) yang nantinya diisi hash.

C.2 Instalasi Laravel Breeze

Laravel Breeze dipasang melalui Composer dengan perintah composer require laravel/breeze. Setelah package tersedia, saya menjalankan php artisan breeze:install blade --no-interaction. Perintah ini membuat route authentication di routes/auth.php, controller di app/Http/Controllers/Auth, request di app/Http/Requests/Auth, view di resources/views/auth, layout guest dan app, halaman profile, serta komponen Blade. Breeze memakai Tailwind CSS dan Vite, jadi asset front-end perlu dikompilasi.

Saya menjalankan npm run build lewat cmd /c karena PowerShell di laptop ini memblokir npm.ps1. Hasil build menulis manifest dan asset ke folder public/build. Setelah itu halaman Breeze bisa tampil dengan CSS yang benar.

Perintah breeze:install menimpa routes/web.php dan resources/views/layouts/app.blade.php. Route LaporBanjir dan view laporan yang sudah ada harus dipasang kembali. Saya menambahkan route laporan ke routes/web.php dan mengubah view laporan agar memakai komponen x-app-layout dari Breeze.

C.3 Route dan Middleware Authentication

Route utama aplikasi ada di routes/web.php. Halaman / menampilkan welcome, /dashboard memakai middleware auth dan verified, route profile memakai middleware auth, dan /laporan juga dimasukkan ke grup auth. Route /laporan/create dan POST /laporan sengaja diletakkan di luar grup auth supaya warga tetap bisa mengirim laporan tanpa login. Di bagian bawah file ada require __DIR__.'/auth.php' yang memuat route authentication.

```php
<?php

use App\Http\Controllers\LaporanController;
use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return view('welcome');
});

Route::get('/dashboard', function () {
    return view('dashboard');
})->middleware(['auth', 'verified'])->name('dashboard');

Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');

    Route::get('/laporan', [LaporanController::class, 'index'])->name('laporan.index');
});

Route::get('/laporan/create', [LaporanController::class, 'create'])->name('laporan.create');
Route::post('/laporan', [LaporanController::class, 'store'])->name('laporan.store');

require __DIR__.'/auth.php';

```

File routes/auth.php membagi route menjadi dua grup. Grup guest berisi register, login, forgot-password, dan reset-password. Grup auth berisi verifikasi email, konfirmasi password, update password, dan logout. Potongan route utama yang saya pakai adalah sebagai berikut.

```php
Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');
    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');
    Route::post('login', [AuthenticatedSessionController::class, 'store']);
});

Route::middleware('auth')->group(function () {
    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});

```

Middleware auth memeriksa session login. Kalau pengguna belum login, Laravel mengarahkan ke route bernama login. Middleware guest melakukan kebalikannya, yaitu mencegah pengguna yang sudah login membuka halaman register dan login. Middleware verified hanya berpengaruh kalau model User mengimplementasikan MustVerifyEmail. Pada project ini baris use Illuminate\Contracts\Auth\MustVerifyEmail masih dikomentari, jadi pengguna baru tetap bisa membuka dashboard tanpa verifikasi email.

C.4 Controller Authentication Breeze

Controller register bawaan Breeze adalah RegisteredUserController. Method create() menampilkan view auth.register. Method store() memvalidasi nama, email unik, dan password yang dikonfirmasi. Password minimal 8 karakter mengikuti Rules\Password::defaults(). Setelah valid, User::create() menyimpan akun dengan Hash::make(), event Registered dipicu, Auth::login() membuat session, dan pengguna diarahkan ke dashboard.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Handle an incoming registration request.
     *
     * @throws ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        Auth::login($user);

        return redirect(route('dashboard', absolute: false));
    }
}

```

Controller login adalah AuthenticatedSessionController. Method create() menampilkan view auth.login. Method store() memanggil LoginRequest untuk memeriksa kredensial, membuat ulang session dengan regenerate(), lalu mengarahkan pengguna ke halaman yang sebelumnya

ingin dibuka atau ke dashboard. Method destroy() memanggil Auth::logout(), mengosongkan session, mengganti token CSRF, dan mengarahkan pengguna ke halaman utama.

```php
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    /**
     * Display the login view.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        return redirect()->intended(route('dashboard', absolute: false));
    }

    /**
     * Destroy an authenticated session.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return redirect('/');
    }
}

```

Pemeriksaan kredensial ada di LoginRequest. Method rules() memastikan email dan password diisi. Method authenticate() memakai Auth::attempt() dengan email, password, dan checkbox remember. Kalau gagal, RateLimiter mencatat percobaan dan ValidationException menampilkan pesan auth.failed. Kalau berhasil, RateLimiter dibersihkan.

```php
public function rules(): array
{
    return [
        'email' => ['required', 'string', 'email'],
        'password' => ['required', 'string'],
    ];
}

public function authenticate(): void
{
    $this->ensureIsNotRateLimited();

    if (! Auth::attempt($this->only('email', 'password'), $this->boolean('remember'))) {
        RateLimiter::hit($this->throttleKey());

        throw ValidationException::withMessages([
            'email' => trans('auth.failed'),
        ]);
    }

    RateLimiter::clear($this->throttleKey());
}

```

Model User menyimpan kolom yang boleh diisi, menyembunyikan password dan remember_token dari serialisasi, serta mengubah kolom password menjadi hash lewat cast hashed.

```php
class User extends Authenticatable
{
    use HasFactory, Notifiable;

    protected $fillable = [
        'name',
        'email',
        'password',
    ];

    protected $hidden = [
        'password',
        'remember_token',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
}

```

C.5 Layout dan View

Breeze memakai dua layout. Layout guest dipakai untuk halaman login dan register, sedangkan layout app dipakai untuk halaman yang sudah login. Layout app memuat navigation, header opsional, dan slot konten.

Navigation saya sesuaikan dengan kebutuhan LaporBanjir. Menu Form Pelaporan selalu tampil karena halaman itu publik. Di dalam directive @auth, menu Dashboard, Daftar

Laporan, dan dropdown Profil Petugas/Logout muncul. Di bagian @else, menu Login dan Register muncul. Dengan begitu tampilan navigasi mengikuti status login pengguna.

```
<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
    <x-nav-link :href="route('laporan.create')" :active="request()->routeIs('laporan.create')">
        {{ __('Form Pelaporan') }}
    </x-nav-link>

    @auth
        <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
            {{ __('Dashboard') }}
        </x-nav-link>

        <x-nav-link :href="route('laporan.index')" :active="request()->routeIs('laporan.index')">
            {{ __('Daftar Laporan') }}
        </x-nav-link>
    @endauth
</div>

<div class="hidden sm:flex sm:items-center sm:ms-6">
    @auth
        <form method="POST" action="{{ route('logout') }}">
            @csrf
            <x-dropdown-link :href="route('logout')">Logout</x-dropdown-link>
        </form>
    @else
        <x-nav-link :href="route('login')">Login</x-nav-link>
        <x-nav-link :href="route('register')">Register</x-nav-link>
    @endauth
</div>

```

Halaman register memakai komponen Blade untuk label, input, pesan galat, dan tombol. Directive @csrf menambahkan token keamanan. Nilai lama diisi kembali lewat old() supaya pengguna tidak perlu mengetik ulang ketika validasi gagal.

```blade
<form method="POST" action="{{ route('register') }}">
    @csrf

    <div>
        <x-input-label for="name" :value="__('Name')" />
        <x-text-input id="name" class="block mt-1 w-full" type="text" name="name" :value="old('name')" required autofocus autocomplete="name" />
        <x-input-error :messages="$errors->get('name')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="email" :value="__('Email')" />
        <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autocomplete="username" />
        <x-input-error :messages="$errors->get('email')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="password" :value="__('Password')" />
        <x-text-input id="password" class="block mt-1 w-full" type="password" name="password" required autocomplete="new-password" />
        <x-input-error :messages="$errors->get('password')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="password_confirmation" :value="__('Confirm Password')" />
        <x-text-input id="password_confirmation" class="block mt-1 w-full" type="password" name="password_confirmation" required autocomplete="new-password" />
        <x-input-error :messages="$errors->get('password_confirmation')" class="mt-2" />
    </div>

    <div class="flex items-center justify-end mt-4">
        <a href="{{ route('login') }}">{{ __('Already registered?') }}</a>
        <x-primary-button class="ms-4">{{ __('Register') }}</x-primary-button>
    </div>
</form>
```

Halaman login memakai pola yang sama, ditambah checkbox Remember me dan tautan lupa password.

```blade
<form method="POST" action="{{ route('login') }}">
    @csrf

    <div>
        <x-input-label for="email" :value="__('Email')" />
        <x-text-input id="email" class="block mt-1 w-full" type="email" name="email" :value="old('email')" required autofocus autocomplete="username" />
        <x-input-error :messages="$errors->get('email')" class="mt-2" />
    </div>

    <div class="mt-4">
        <x-input-label for="password" :value="__('Password')" />
        <x-text-input id="password" class="block mt-1 w-full" type="password" name="password" required autocomplete="current-password" />
        <x-input-error :messages="$errors->get('password')" class="mt-2" />
    </div>

    <div class="block mt-4">
        <label for="remember_me" class="inline-flex items-center">
            <input id="remember_me" type="checkbox" name="remember">
            <span class="ms-2 text-sm text-gray-600">{{ __('Remember me') }}</span>
        </label>
    </div>

    <div class="flex items-center justify-end mt-4">
        <a href="{{ route('password.request') }}">{{ __('Forgot your password?') }}</a>
        <x-primary-button class="ms-3">{{ __('Log in') }}</x-primary-button>
    </div>
</form>

```

View laporan dari Modul 4 saya ubah agar memakai x-app-layout dan kelas Tailwind. Form pelaporan tetap berisi nama pelapor, lokasi, tinggi genangan, dan tanggal kejadian. Validasi input ditambahkan di LaporanController supaya data yang masuk tidak kosong.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;

class LaporanController extends Controller
{
    public function index()
    {
        $laporan = Laporan::latest()->get();

        return view('laporan.index', compact('laporan'));
    }

    public function create()
    {
        return view('laporan.form');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama_pelapor' => ['required', 'string', 'max:255'],
            'lokasi' => ['required', 'string', 'max:255'],
            'tinggi_genangan' => ['required', 'integer', 'min:1'],
            'tanggal_kejadian' => ['required', 'date'],
        ]);

        $laporan = Laporan::create($validated);

        return view('laporan.konfirmasi', compact('laporan'));
    }
}

```

```blade
<form action="{{ route('laporan.store') }}" method="POST" class="space-y-6">
    @csrf

    <div>
        <x-input-label for="nama_pelapor" :value="__('Nama Pelapor')" />
        <x-text-input id="nama_pelapor" name="nama_pelapor" type="text" class="mt-1 block w-full" :value="old('nama_pelapor')" required autofocus />
        <x-input-error :messages="$errors->get('nama_pelapor')" class="mt-2" />
    </div>

    <div>
        <x-input-label for="lokasi" :value="__('Lokasi Kejadian')" />
        <x-text-input id="lokasi" name="lokasi" type="text" class="mt-1 block w-full" :value="old('lokasi')" required />
        <x-input-error :messages="$errors->get('lokasi')" class="mt-2" />
    </div>

    <div>
        <x-input-label for="tinggi_genangan" :value="__('Tinggi Genangan Air (cm)')" />
        <x-text-input id="tinggi_genangan" name="tinggi_genangan" type="number" min="1" class="mt-1 block w-full" :value="old('tinggi_genangan')" required />
        <x-input-error :messages="$errors->get('tinggi_genangan')" class="mt-2" />
    </div>

    <div>
        <x-input-label for="tanggal_kejadian" :value="__('Tanggal Kejadian')" />
        <x-text-input id="tanggal_kejadian" name="tanggal_kejadian" type="date" class="mt-1 block w-full" :value="old('tanggal_kejadian')" required />
        <x-input-error :messages="$errors->get('tanggal_kejadian')" class="mt-2" />
    </div>

    <x-primary-button>{{ __('Kirim Laporan') }}</x-primary-button>
</form>
```

C.6 Seeder Petugas dengan Faker

Setelah tabel users tersedia, saya membuat PetugasSeeder. Seeder ini memakai fake('id_ID') supaya nama yang dihasilkan sesuai locale Indonesia. Akun admin BPBD dibuat dengan email admin@bpbd.go.id dan password admin12345. Lima akun petugas dibuat dengan email petugas1@bpbd.go.id sampai petugas5@bpbd.go.id, nama dari Faker, dan password petugas123. Semua password disimpan lewat Hash::make(). Saya memakai updateOrCreate supaya seeder aman dijalankan ulang tanpa membuat email ganda.

```php
<?php

namespace Database\Seeders;

use App\Models\User;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class PetugasSeeder extends Seeder
{
    public function run(): void
    {
        $faker = fake('id_ID');

        User::updateOrCreate(
            ['email' => 'admin@bpbd.go.id'],
            [
                'name' => 'Admin BPBD',
                'password' => Hash::make('admin12345'),
            ]
        );

        for ($i = 1; $i <= 5; $i++) {
            User::updateOrCreate(
                ['email' => 'petugas'.$i.'@bpbd.go.id'],
                [
                    'name' => $faker->name(),
                    'password' => Hash::make('petugas123'),
                ]
            );
        }
    }
}

```

DatabaseSeeder memanggil PetugasSeeder dan LaporanSeeder. LaporanSeeder dari Modul 4 saya pertahankan, tetapi saya tambahkan pengecekan supaya data laporan tidak bertambah setiap kali seeder dijalankan.

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    use WithoutModelEvents;

    public function run(): void
    {
        $this->call([
            PetugasSeeder::class,
            LaporanSeeder::class,
        ]);
    }
}

```

Seeder dijalankan dengan php artisan db:seed --class=PetugasSeeder. Setelah itu saya memeriksa tabel users di phpMyAdmin. Ada enam akun petugas, yaitu satu admin dan lima petugas dummy. Kolom password berisi hash bcrypt dengan awalan $2y$12$, bukan teks asli.

C.7 Menjalankan Aplikasi dan Menguji Alur

Aplikasi dijalankan dengan php artisan serve. Apache dan MySQL pada XAMPP tetap berjalan karena database db_laporbanjir dipakai oleh aplikasi. Untuk pengujian, saya membuka /laporan tanpa login. Middleware auth mengarahkan saya ke /login. Setelah itu saya membuka /register dan mencoba mengisi password pendek dengan konfirmasi yang berbeda untuk melihat pesan validasi. Setelah data diperbaiki, registrasi berhasil dan saya diarahkan ke dashboard.

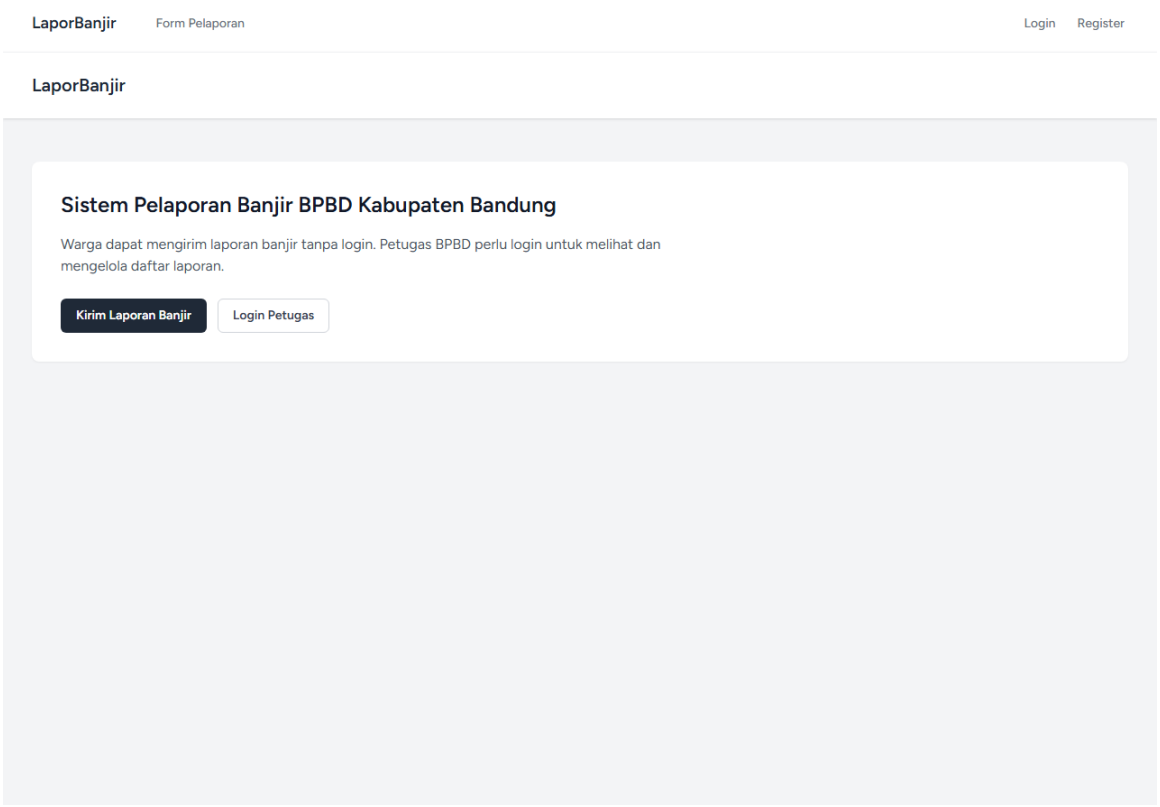
Saya membuka halaman profile, melihat informasi akun, lalu logout. Setelah logout, /laporan kembali diarahkan ke /login. Saya juga mencoba login dengan password salah untuk melihat pesan galat, lalu login memakai akun admin hasil seeder. Terakhir saya membuka form

pelaporan sebagai pengguna tanpa login dan mengirim satu laporan. Form tetap bisa diakses dan laporan masuk ke tabel laporan.

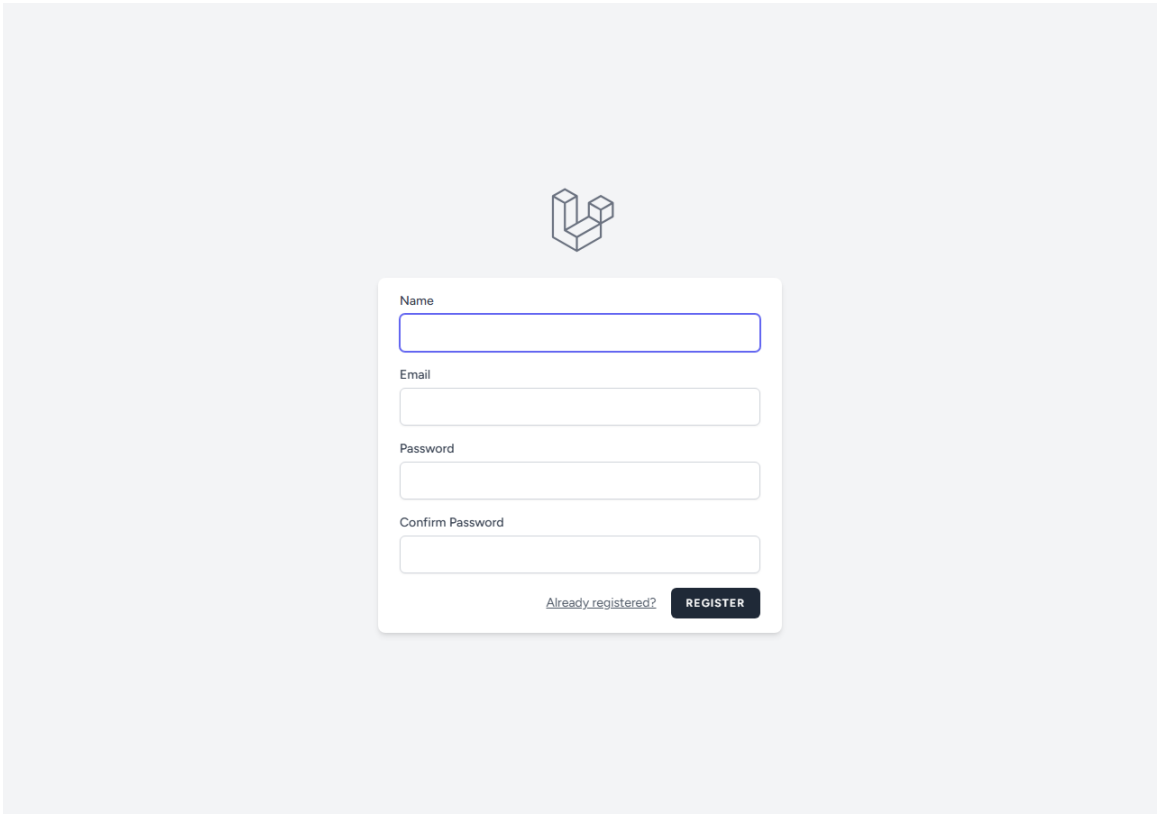
Hasil setiap pengujian ada pada bagian D.

D. Hasil dan Dokumentasi Program

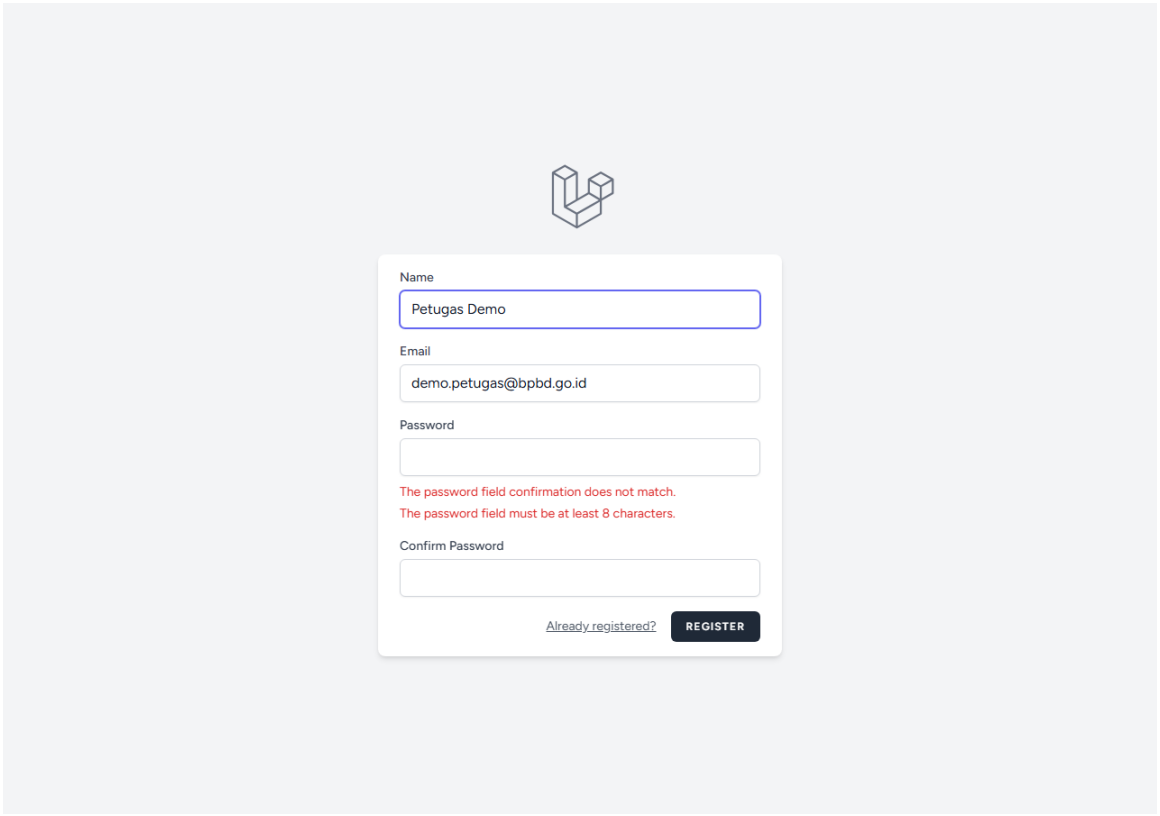
Berikut tangkapan layar dari aplikasi LaporBanjir yang sudah dijalankan dan dari phpMyAdmin.



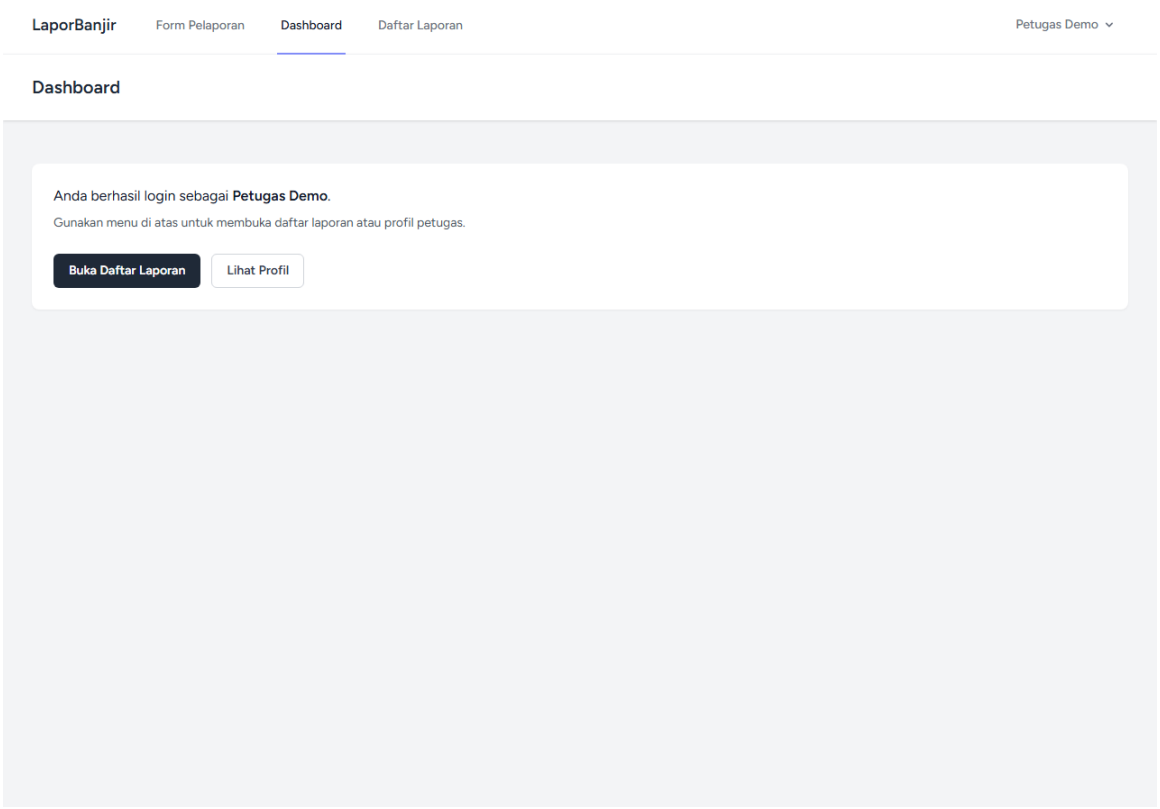
Gambar 1. Halaman awal LaporBanjir dengan menu Form Pelaporan, Login, dan Register.



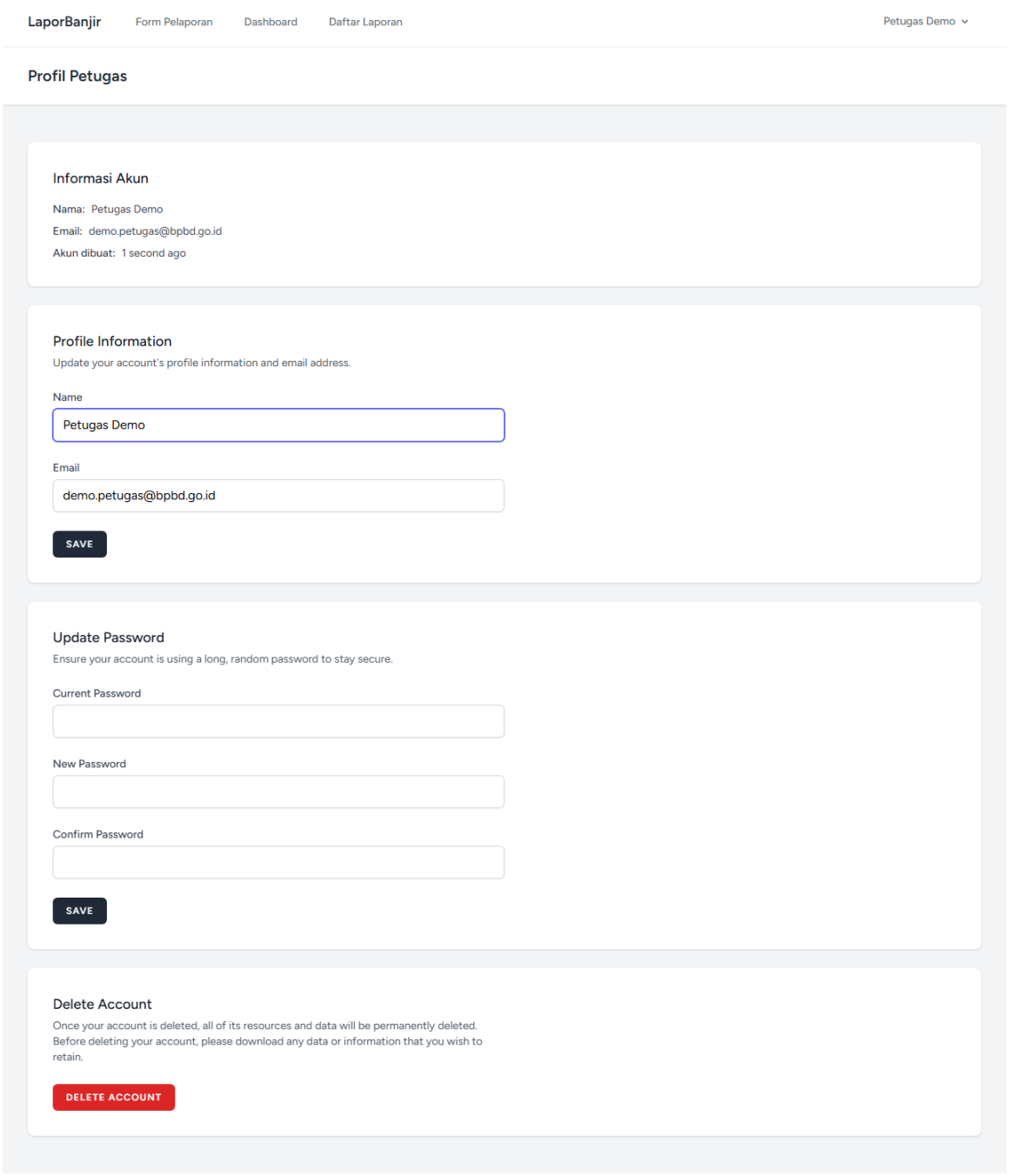
Gambar 2. Halaman register sebelum diisi.



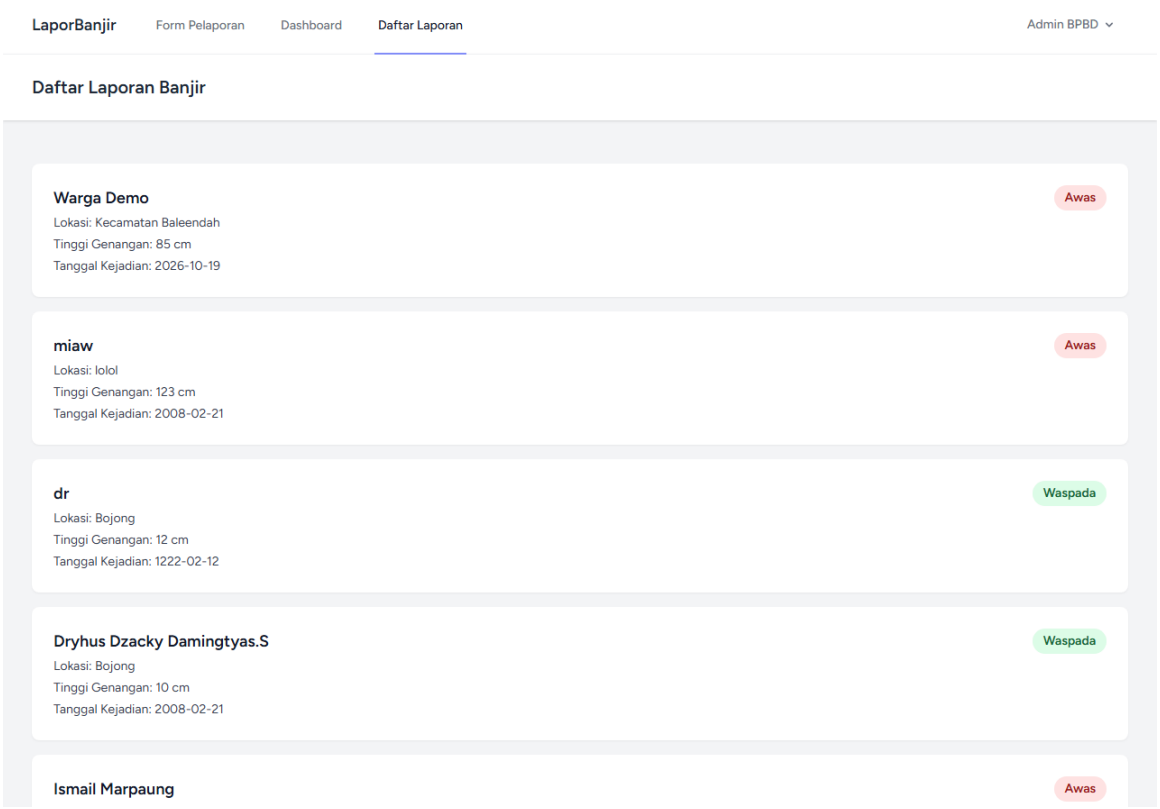
Gambar 3. Pesan galat validasi saat password kurang dari 8 karakter dan konfirmasi tidak sama.



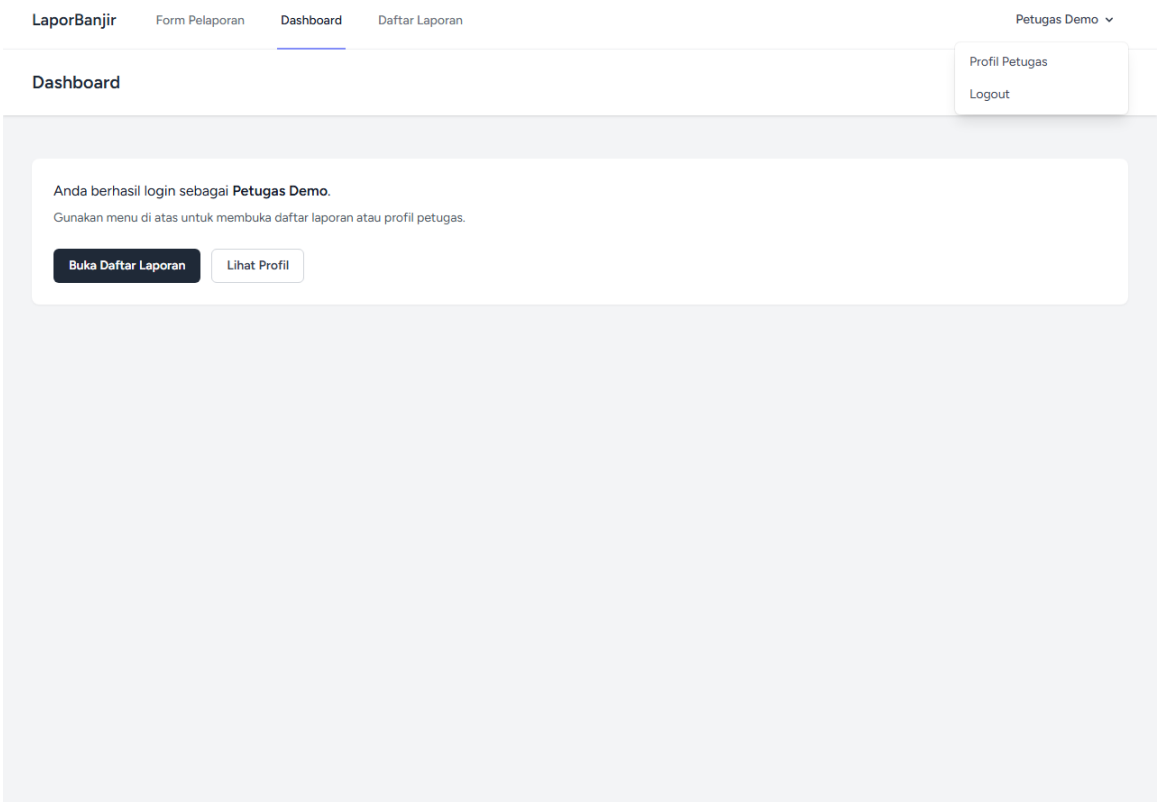
Gambar 4. Dashboard setelah registrasi berhasil. Nama Petugas Demo muncul di navigasi.



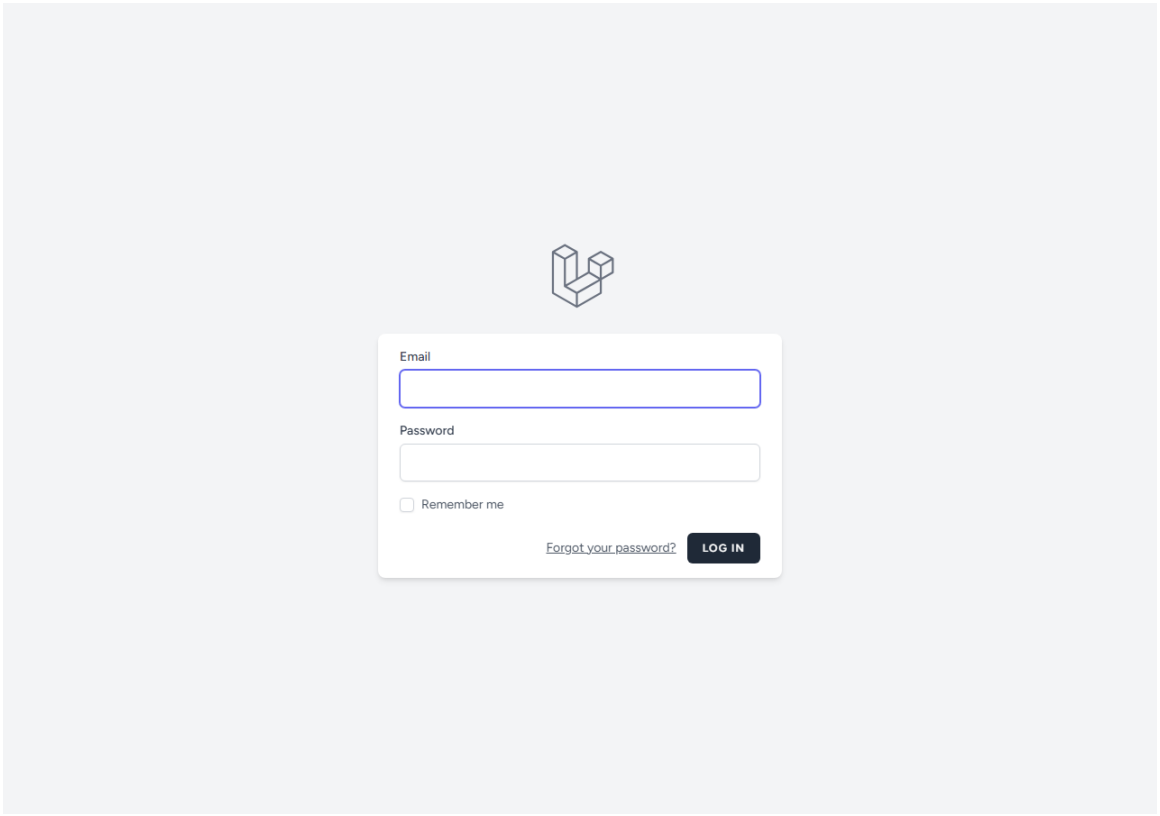
Gambar 5. Halaman profil petugas dengan informasi akun.



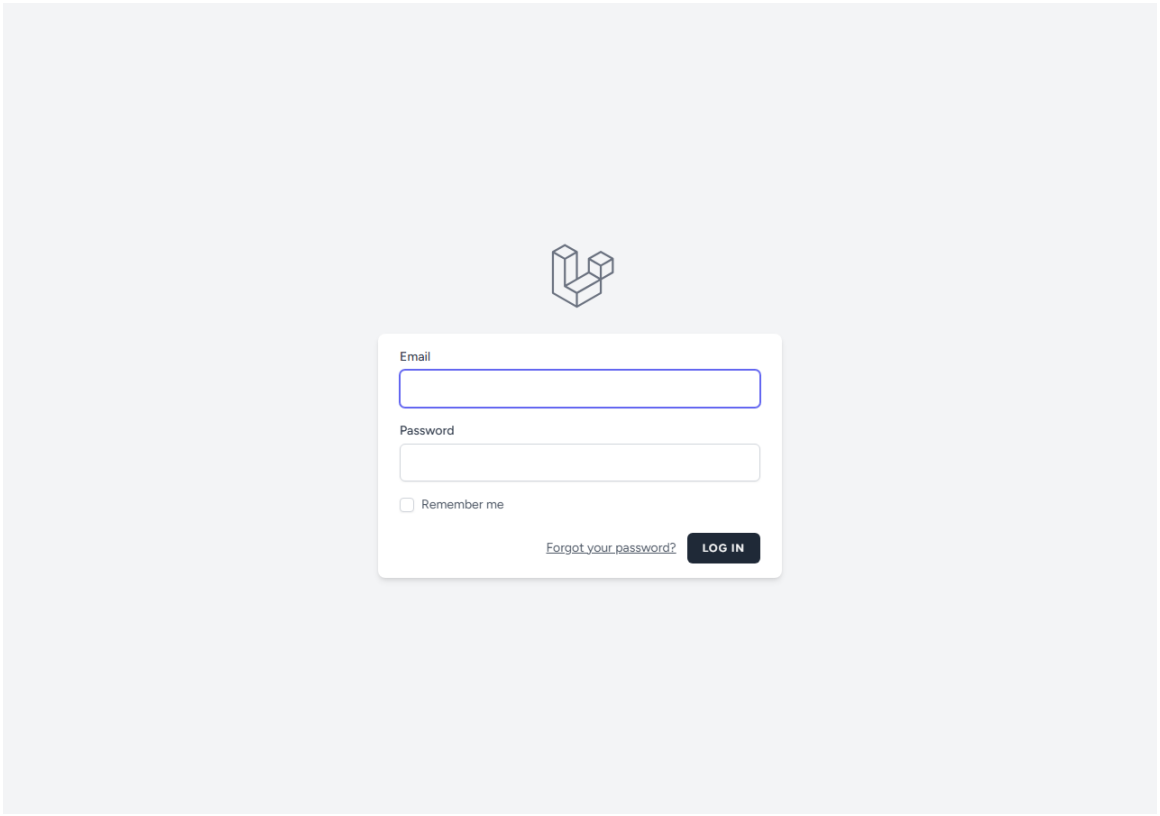
Gambar 6. Daftar laporan yang hanya bisa dibuka setelah login.



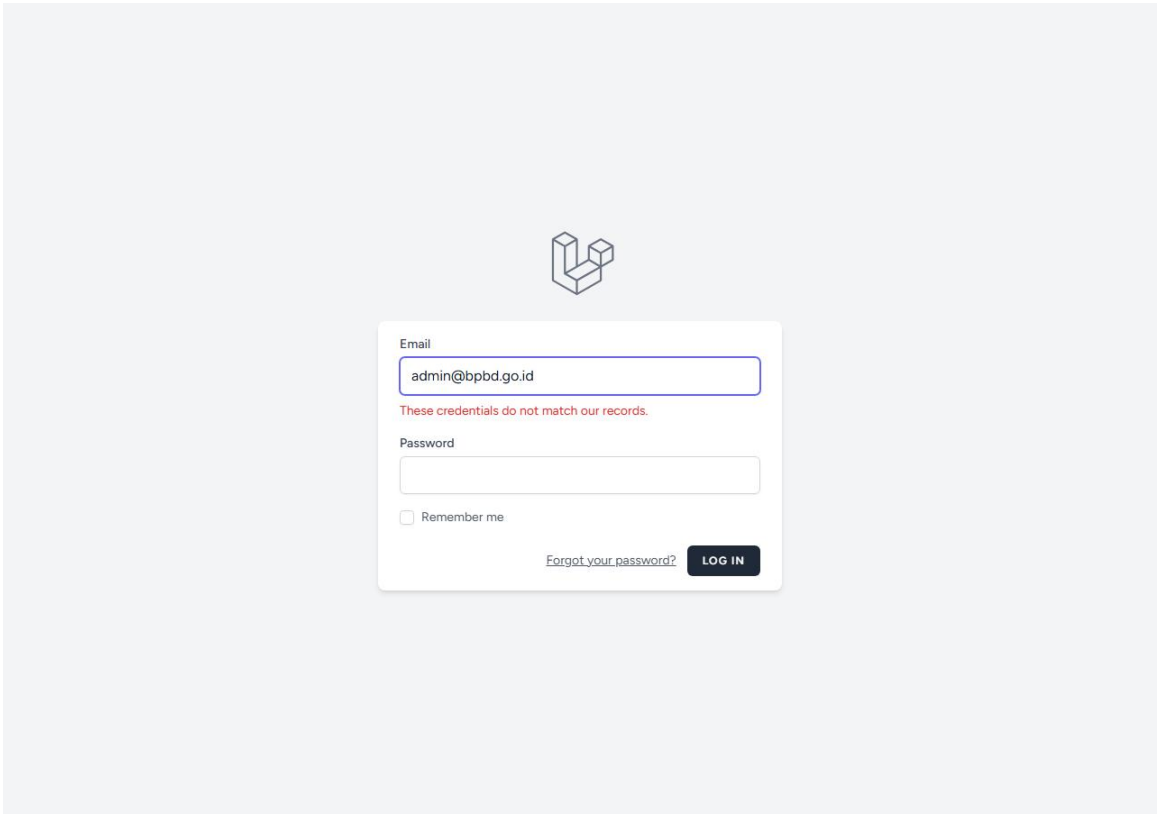
Gambar 7. Menu dropdown berisi Profil Petugas dan Logout.



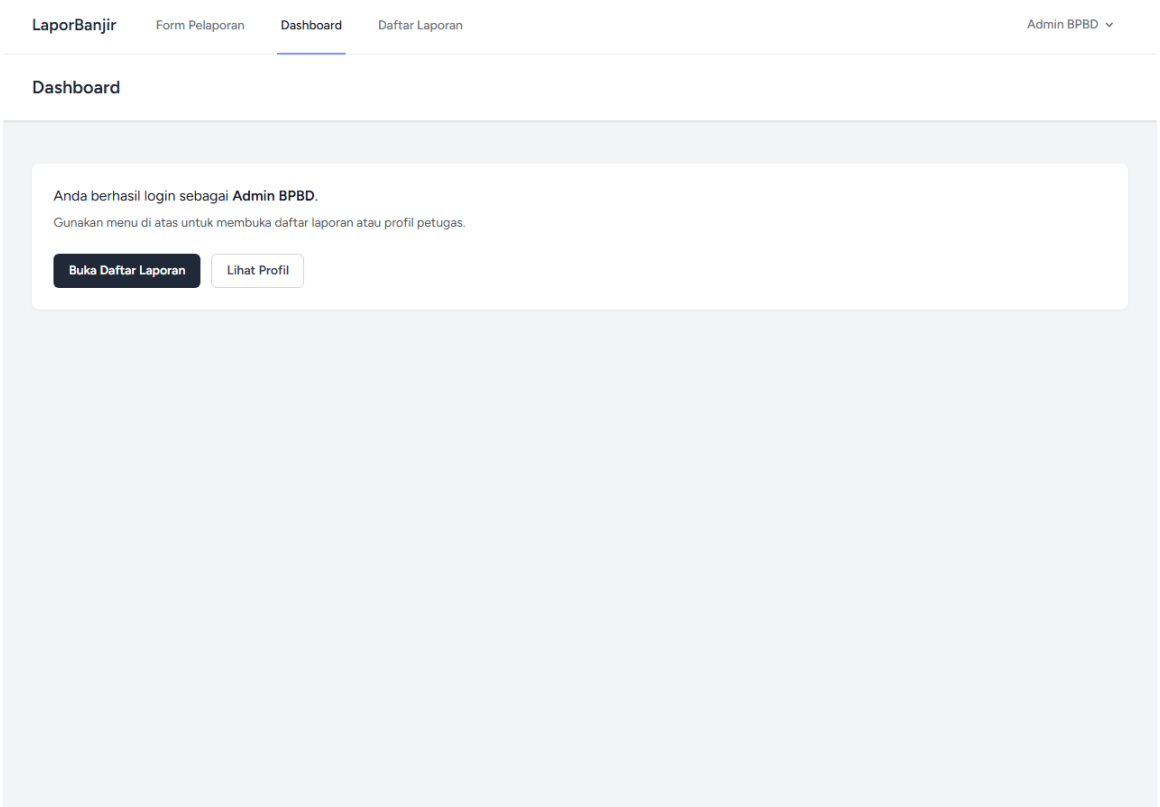
Gambar 8. Halaman login yang muncul setelah /laporan dibuka tanpa login.



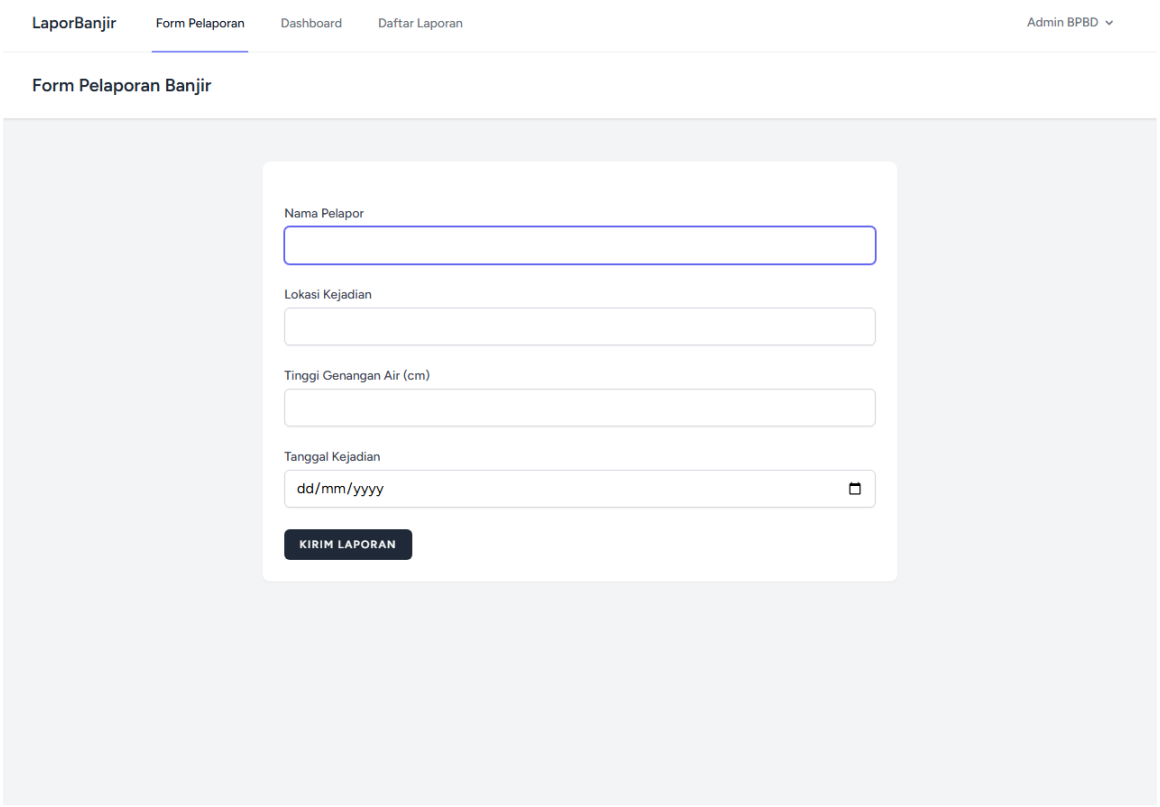
Gambar 9. Halaman login.



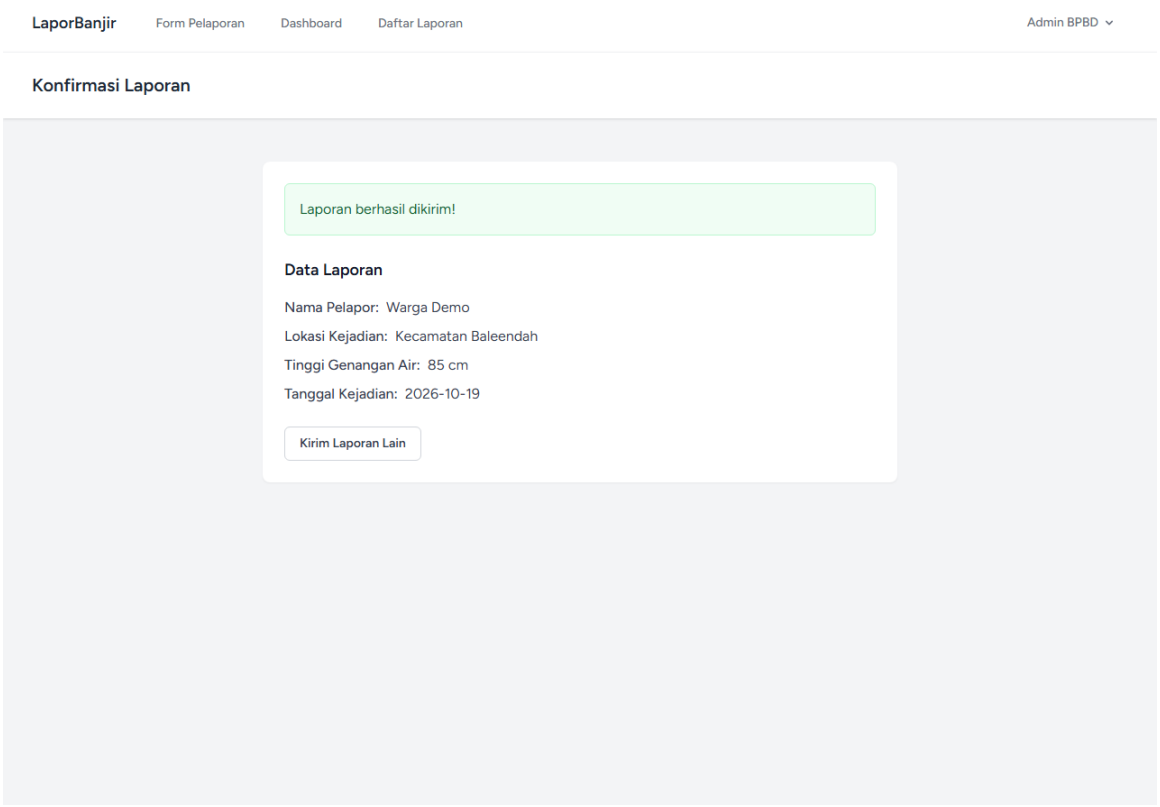
Gambar 10. Pesan galat These credentials do not match our records. saat password salah.



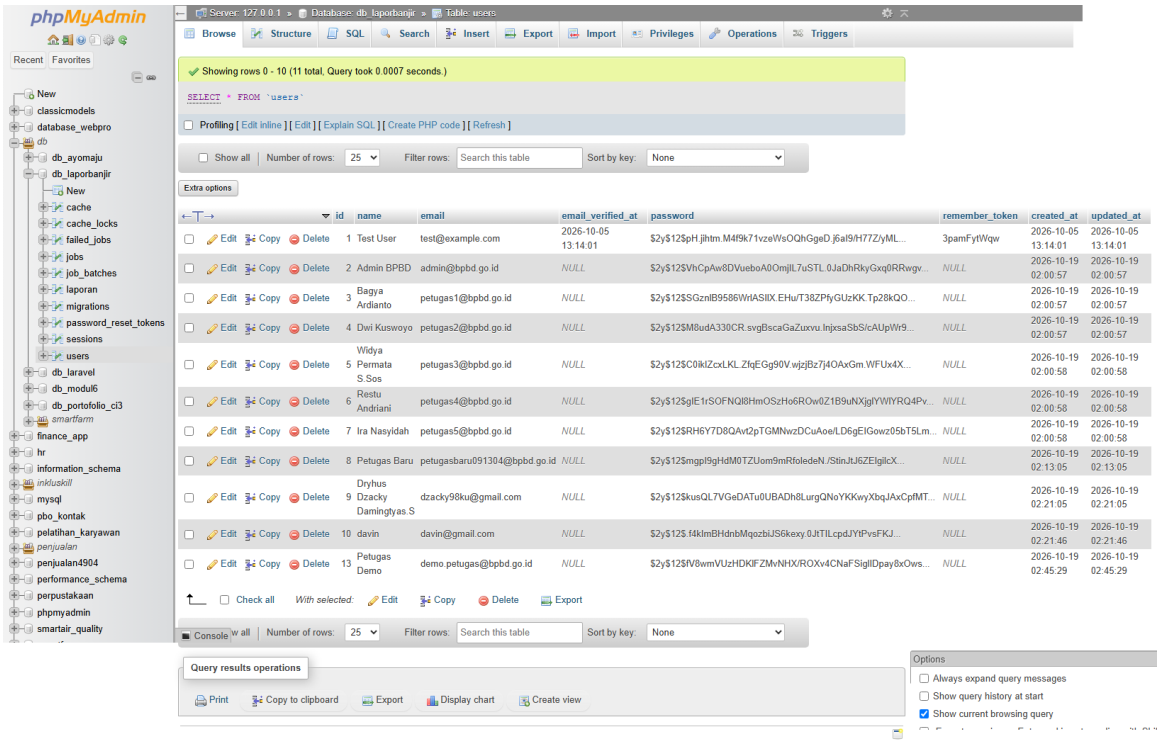
Gambar 11. Dashboard admin BPBD setelah login memakai akun hasil seeder.



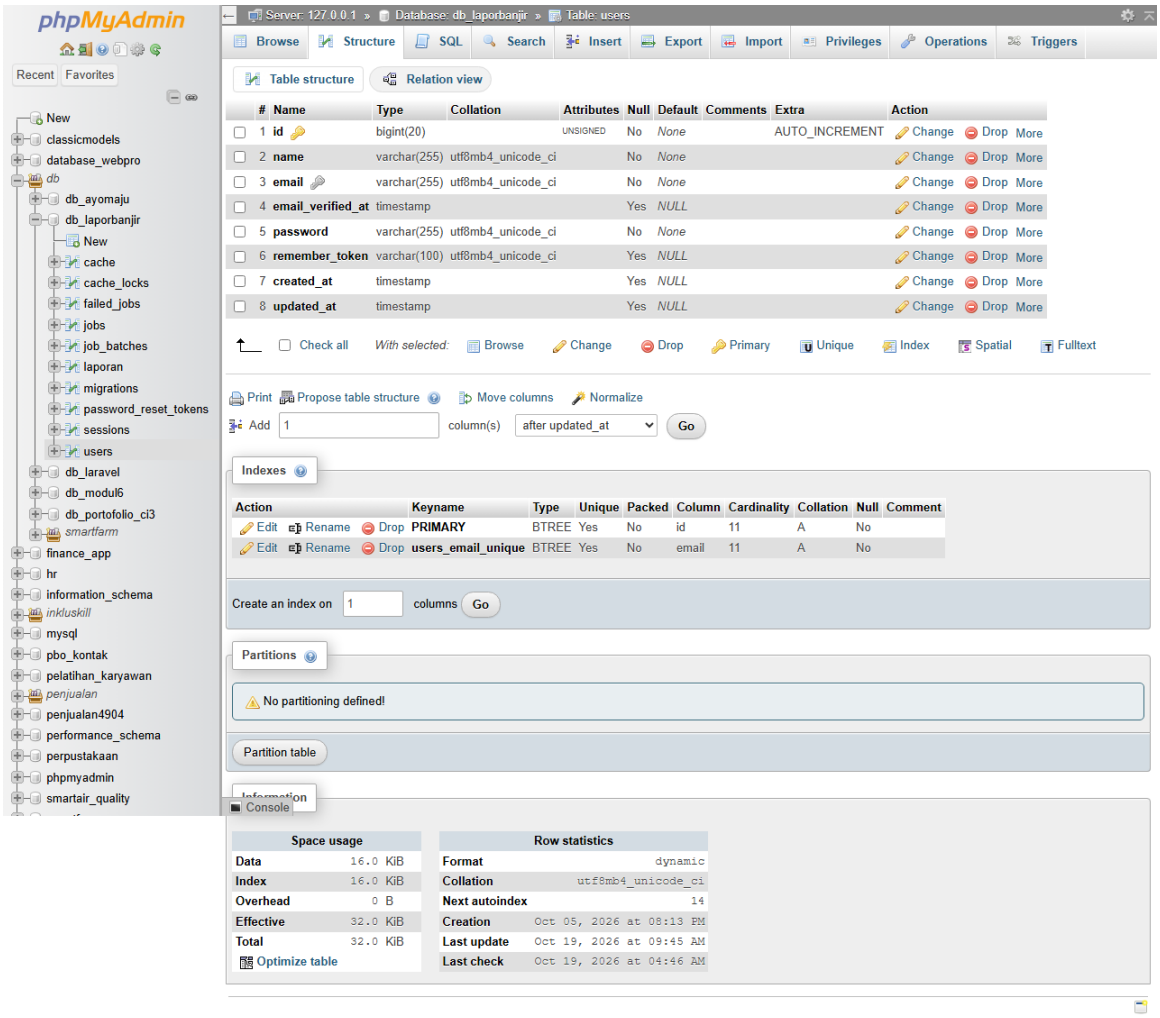
Gambar 12. Form pelaporan yang tetap bisa diakses tanpa login.



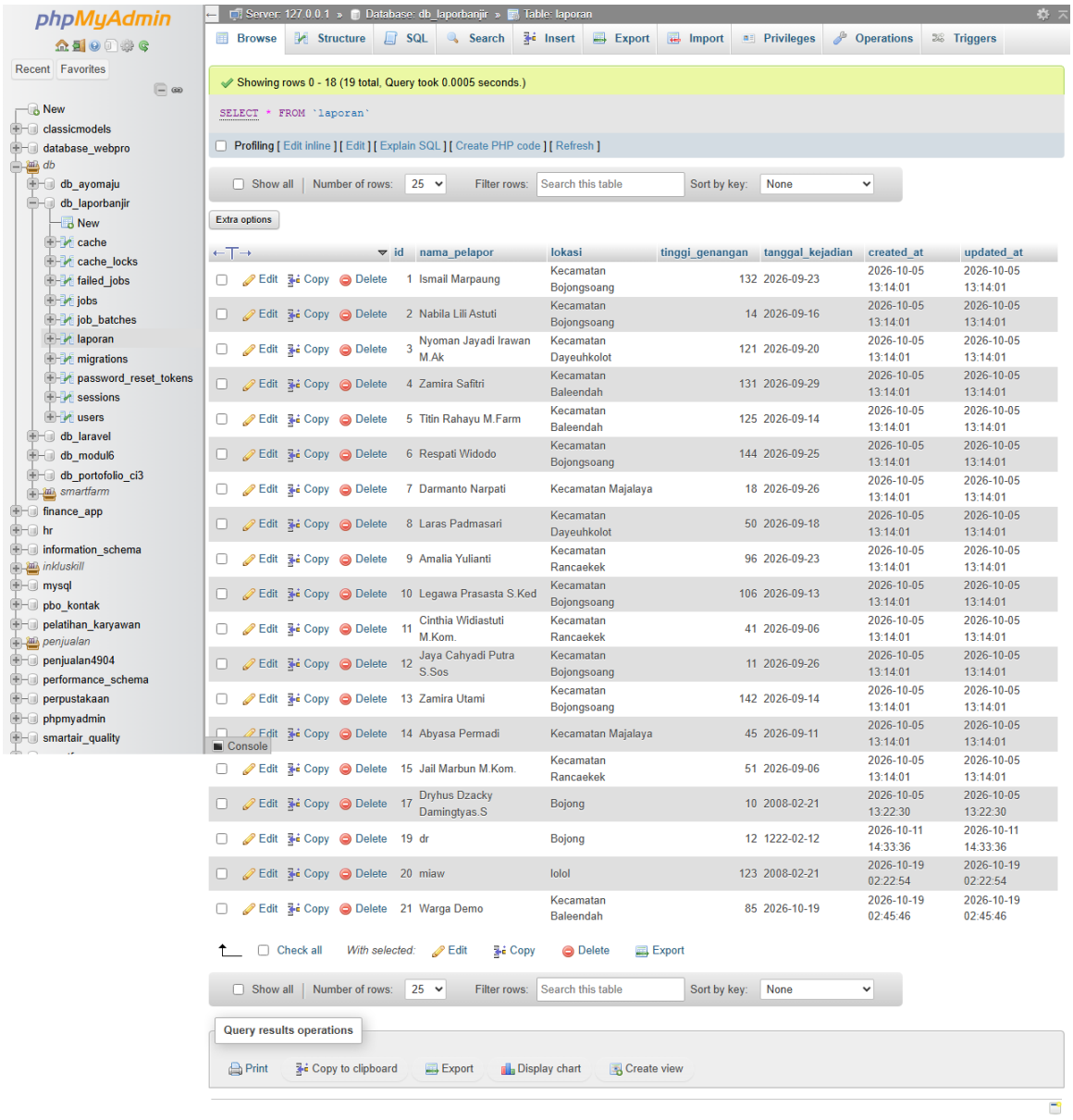
Gambar 13. Konfirmasi laporan setelah form dikirim.



Gambar 14. Tabel users di phpMyAdmin. Kolom password berisi hash dengan awalan $2y$12$.



Gambar 15. Struktur tabel users. Kolom password bertipe varchar(255).



Gambar 16. Tabel laporan di phpMyAdmin setelah laporan baru dikirim.

E. Analisis dan Pembahasan

E.1 Peran Authentication, Session, dan Middleware

Authentication memastikan bahwa pengguna yang mengakses aplikasi adalah orang yang ia akui. Pada project ini proses itu terjadi di halaman login. Setelah email dan password cocok, Laravel menyimpan status login di session. Browser hanya menyimpan cookie session, sedangkan data session disimpan di server. Project ini memakai SESSION_DRIVER=database, sehingga data session tersimpan di tabel sessions.

Setiap request berikutnya membawa cookie session. Middleware auth membaca session itu dan memeriksa apakah pengguna sudah login. Kalau sudah, request diteruskan ke controller atau view. Kalau belum, pengguna diarahkan ke route login. Middleware guest bekerja sebaliknya, yaitu mencegah pengguna yang sudah login membuka halaman register dan login. Pada project ini /dashboard dan /laporan dilindungi auth, sedangkan /laporan/create dan POST /laporan tetap terbuka untuk warga.

Middleware verified tidak otomatis mewajibkan verifikasi email. Middleware itu hanya memeriksa apakah pengguna sudah memverifikasi email jika model User mengimplementasikan MustVerifyEmail. Pada project ini interface tersebut masih dikomentari, jadi pengguna baru langsung bisa membuka dashboard. Perilaku ini sesuai bawaan Breeze dan tidak mengubah alur login.

E.2 Hashing Password

Password tidak disimpan dalam bentuk teks asli. Saat register, RegisteredUserController memanggil Hash::make() sebelum menyimpan password. Saat login, Auth::attempt() mencocokkan password yang diketik dengan hash di database memakai Hash::check(). Karena itu password asli tidak pernah tersimpan dan tidak bisa dibaca langsung dari tabel users.

Hash yang muncul di phpMyAdmin berawalan $2y$12$. Awalan itu menunjukkan algoritma bcrypt dengan cost 12. Bcrypt menambahkan salt dan sengaja dibuat lambat supaya percobaan menebak password secara massal lebih sulit. Model User juga menyembunyikan password dan remember_token lewat properti $hidden, serta memakai cast hashed untuk kolom password. Dengan pengaturan ini, password tetap aman walaupun isi database dilihat orang lain.

E.3 Penerapan Breeze pada LaporBanjir

Breeze menyiapkan hampir semua kebutuhan authentication secara otomatis. Route register, login, logout, forgot-password, reset-password, verifikasi email, dan profile sudah dibuat. Controller, view, layout, dan file pengujian juga sudah tersedia. Pekerjaan yang tersisa adalah menyesuaikan database, mengompilasi asset, menghubungkan route LaporBanjir, dan menguji alurnya. Cara ini cepat dipakai untuk project yang ingin segera punya fitur login.

Karena Breeze menimpa routes/web.php dan layouts/app.blade.php, integrasi dengan project lama tidak bisa dilakukan tanpa penyesuaian. Saya harus menambahkan kembali route

laporan dan mengubah view laporan agar memakai x-app-layout. View laporan dari Modul 4 yang memakai @extends('layouts.app') tidak lagi cocok karena layout Breeze memakai slot komponen, bukan section Blade.

Pada praktikum ini saya memakai Breeze sebagai mekanisme authentication utama. Versi manual yang sempat dibuat sebelumnya tidak dipakai di versi akhir karena akan bertabrakan dengan route login dan register dari Breeze. Konsep yang dipakai tetap sama, yaitu validasi input, Hash::make(), Auth::attempt(), Auth::logout(), session, dan middleware auth.

E.4 Alur Register, Login, dan Logout

Alur register dimulai dari GET /register. Route memanggil RegisteredUserController@create dan menampilkan form. Saat form dikirim dengan POST, method store() memvalidasi nama, email, dan password. Kalau valid, User::create() menyimpan akun dengan password hash, event Registered dipicu, Auth::login() membuat session, dan pengguna diarahkan ke /dashboard.

Alur login dimulai dari GET /login yang menampilkan form. Saat form dikirim dengan POST, LoginRequest memvalidasi input lalu memanggil Auth::attempt(). Kalau kredensial cocok, session dibuat ulang dengan regenerate() untuk mencegah session fixation, lalu pengguna diarahkan ke halaman yang sebelumnya ingin dibuka atau ke dashboard. Kalau gagal, pengguna kembali ke halaman login dengan pesan galat.

Alur logout dimulai dari tombol Logout di menu pengguna. Tombol itu mengirim POST /logout. AuthenticatedSessionController@destroy memanggil Auth::logout(), mengosongkan session dengan invalidate(), mengganti token CSRF dengan regenerateToken(), lalu mengarahkan pengguna ke halaman utama. Setelah itu halaman /laporan tidak bisa dibuka lagi sampai pengguna login kembali.

E.5 Perlindungan Halaman dan Navigasi

Halaman daftar laporan adalah halaman pengelolaan yang hanya boleh dilihat petugas. Karena itu route /laporan dimasukkan ke grup middleware auth. Bukti pengujiannya ada pada Gambar 8, yaitu /laporan yang dibuka tanpa login langsung diarahkan ke halaman login. Sebaliknya, form pelaporan tetap publik karena warga tidak diwajibkan punya akun untuk melapor.

Navigasi memakai directive @auth dan @else. Pengguna yang belum login melihat menu Form Pelaporan, Login, dan Register. Pengguna yang sudah login melihat menu Form

Pelaporan, Dashboard, Daftar Laporan, serta dropdown Profil Petugas dan Logout. Dengan cara ini menu yang tampil sesuai dengan hak akses pengguna dan tidak ada tautan yang mengarah ke halaman yang belum tersedia.

E.6 Seeder dan Data Dummy

PetugasSeeder memakai Faker dengan locale id_ID untuk menghasilkan nama petugas. Lima akun dummy dibuat dengan email yang tetap supaya mudah diuji, yaitu petugas1@bpbd.go.id sampai petugas5@bpbd.go.id. Akun admin memakai email admin@bpbd.go.id. Semua password di-hash dengan Hash::make(), sehingga tabel users tidak menyimpan password asli.

Saya memakai updateOrCreate agar seeder bisa dijalankan ulang tanpa membuat akun ganda. DatabaseSeeder memanggil PetugasSeeder dan LaporanSeeder, jadi php artisan db:seed menjalankan keduanya. LaporanSeeder dari Modul 4 diberi pengecekan agar tidak menambah data laporan baru kalau tabel laporan sudah berisi data. Data laporan lama dari Modul 4 tetap dipakai.

E.7 Kendala dan Penyelesaian

Ada tiga kendala yang saya temui selama mengerjakan project ini.

1. Perintah breeze:install blade menimpa routes/web.php dan resources/views/layouts/app.blade.php. Route LaporBanjir dan view laporan sempat tidak terbaca. Saya menambahkan kembali route laporan ke routes/web.php dan mengubah view laporan agar memakai x-app-layout dari Breeze.
2. Perintah npm diblokir oleh kebijakan eksekusi PowerShell. Muncul pesan npm.ps1 cannot be loaded because running scripts is disabled on this system. Saya menjalankan build lewat cmd /c npm run build, sehingga Vite tetap bisa mengompilasi asset.
3. Database db_laporbanjir sudah berisi tabel dari Modul 4, jadi php artisan migrate menampilkan Nothing to migrate. Saya tetap memakai tabel yang sudah ada dan menyalin migration laporan dengan nama file yang sama supaya status migration tetap konsisten. Kalau database kosong, migration yang sama akan membuat tabel users dan laporan dari awal.

E.8 Catatan Perbedaan dengan Modul

Ada beberapa perbedaan antara project ini dan contoh pada modul. Perbedaan ini saya catat supaya hasil praktikum tidak dibaca sebagai hal yang sama persis dengan contoh.

1. Tugas praktikum meminta authentication manual dengan RegisterController dan LoginController. Pada project ini saya memakai Breeze karena Breeze sudah diminta untuk dipasang dan package-nya sudah tersedia. Kode manual yang sempat dibuat tidak dipakai di versi akhir agar tidak ada dua route login dan register yang bertabrakan. Konsep hashing, session, dan middleware tetap sama.
2. Project yang dipakai adalah lanjutan LaporBanjir dari Modul 4, bukan project baru. Database yang dipakai tetap db_laporbanjir. Halaman yang dilindungi adalah /laporan, sedangkan /laporan/create tetap publik.
3. PetugasSeeder memakai locale id_ID dan membuat lima akun dummy plus satu akun admin, sesuai ketentuan modul. Seeder dijalankan lewat DatabaseSeeder dan juga bisa dijalankan langsung dengan --class=PetugasSeeder.
4. LaporanSeeder dari Modul 4 dipertahankan dan dibuat idempotent. Tujuannya supaya data laporan lama tidak hilang dan tidak bertambah setiap kali seeder dijalankan.

F. Kesimpulan

1. Laravel Breeze 2.4.2 berhasil dipasang dan menyediakan route, controller, view, serta layout untuk register, login, logout, reset password, verifikasi email, dan profile.
2. Project LaporBanjir dari Modul 4 berhasil dihubungkan dengan Breeze memakai database db_laporbanjir tanpa membuat ulang data laporan lama.
3. Alur authentication berjalan dengan baik. Register menyimpan akun dengan password hash, login memeriksa kredensial lewat Auth::attempt(), session menyimpan status login, dan logout menghapus session.
4. Middleware auth melindungi /laporan dan /dashboard. Pengguna yang belum login diarahkan ke /login, sedangkan form pelaporan tetap bisa diakses tanpa login.
5. PetugasSeeder berhasil membuat lima akun petugas dummy dan satu akun admin. Password tersimpan sebagai hash bcrypt dengan awalan $2y$12$, dan tabel users di phpMyAdmin menunjukkan bukti tersebut.
6. Navigasi memakai directive @auth dan @else, sehingga menu Login dan Register tampil saat belum login, sedangkan menu Daftar Laporan, Profil Petugas, dan Logout tampil setelah login.
7. Breeze mempercepat pembuatan authentication, tetapi menimpa route dan layout yang sudah ada. Integrasi dengan project lama tetap memerlukan penyesuaian route dan view.

G. Link Repository Github

Kode program lengkap praktikum ini dapat diakses pada repositori GitHub berikut:

https://github.com/Dryhus21/PABW.git